

La Guilde des Aventuriers

Construire l'API de gestion d'une guilde avec Express + Prisma

Cours : **Service Web (25604)** • Travail pratique

Stack : Node.js, TypeScript, Express 5, Prisma 7, PostgreSQL (Neon)

Le scénario. La *Guilde des Aventuriers* déborde de héros, de quêtes et de trésors... mais tout est encore noté sur de vieux parchemins ! Le Maître de Guilde vous engage comme **Développeur · euse du Royaume** pour construire une vraie API REST qui gèrera les aventuriers et leurs quêtes dans une base de données PostgreSQL.

À chaque étape réussie, vous gagnez des points d'expérience (**XP**). Atteignez le rang **Légendaire** en terminant tout le contrat !

Objectifs pédagogiques

- Initialiser un projet Node.js + TypeScript et installer les dépendances.
- Modéliser des données réelles avec Prisma, y compris des **enums** et une relation 1-N.
- Connecter Prisma à PostgreSQL (Neon) via l'adaptateur `@prisma/adaptor-neon`.
- Construire un backend Express avec des **routers** et les verbes GET, POST, DELETE.
- Insérer puis tester des données réelles, et prouver leur persistance.

1. Quête 1 — Préparer le campement (installation)

[100 XP]

Vous travaillez dans un nouveau dossier `guilde-api`. Ouvrez un terminal PowerShell dedans.

À faire

1. Initialiser le projet et installer toutes les dépendances ci-dessous.
2. Initialiser la configuration TypeScript.

Listing 1 – Initialisation & dépendances

```
1 # 1. Creer le projet Node
2 npm init -y
3
4 # 2. Dependances de production
5 npm install express dotenv @prisma/client @prisma/adaptor-neon
6
7 # 3. Dependances de developpement
8 npm install --save-dev typescript tsx @types/node @types/express prisma
9
10 # 4. Generer un tsconfig.json
11 npx tsc --init
```

Rappel des outils : `tsx` exécute du TypeScript sans compilation manuelle (idéal en dev), `prisma` est l'outil en ligne de commande (CLI), `@prisma/adaptor-neon` permet à Prisma 7 de

parler à une base **Neon**.

Ajoutez ce script dans la section `scripts` de votre `package.json` :

Listing 2 – `package.json` (extrait)

```
1 "scripts": {  
2   "dev": "tsx watch src/server.ts",  
3   "seed": "tsx prisma/seed.ts"  
4 }
```

2. Quête 2 — Ouvrir le portail vers la base (Neon + .env)

[100 XP]

À faire

1. Créez (gratuitement) une base PostgreSQL sur neon.tech et copiez sa chaîne de connexion.
2. À la racine du projet, créez un fichier `.env` :

Listing 3 – `.env`

```
1 DATABASE_URL="postgresql://utilisateur:motdepasse@hote.neon.tech/nomdb?sslmode=  
   require"  
2 PORT=3000
```

Piège ! Ne committez **jamais** le fichier `.env` sur Git. Créez un `.gitignore` contenant au minimum : `node_modules/`, `.env` et `generated/`.

3. Quête 3 — Dessiner la carte du royaume (schéma Prisma)

[250 XP]

C'est le cœur de l'exercice : un **beau** modèle de données. On modélise des **Aventuriers** qui acceptent des **Quêtes** (relation 1–N), avec des **enums** pour la classe, le rang et la difficulté.

À faire

1. Initialisez Prisma : `npx prisma init`.
2. Remplacez le contenu de `prisma/schema.prisma` par le schéma ci-dessous.

Listing 4 – `prisma/schema.prisma`

```
1 generator client {  
2   provider = "prisma-client"  
3   output   = "../generated/prisma"  
4 }  
5  
6 datasource db {  
7   provider = "postgresql"  
8 }  
9  
10 // ----- ENUMS -----  
11 enum Classe {  
12   GUERRIER  
13   MAGE  
14   VOLEUR
```

```

15   CLERC
16   RODEUR
17 }
18
19 enum Rang {
20   BRONZE
21   ARGENT
22   OR
23   PLATINE
24   LEGENDAIRE
25 }
26
27 enum Difficulte {
28   FACILE
29   NORMALE
30   DIFFICILE
31   CAUCHEMAR
32 }
33
34 enum StatutQuete {
35   DISPONIBLE
36   EN_COURS
37   TERMINEE
38   ECHOUEE
39 }
40
41 // ----- MODELES -----
42 model Aventurier {
43   id          Int          @id @default(autoincrement())
44   pseudo      String       @unique
45   email       String       @unique
46   classe      Classe       @default(GUERRIER)
47   rang        Rang         @default(BRONZE)
48   niveau      Int          @default(1)
49   piecesOr    Int          @default(0)
50   createdAt   DateTime     @default(now())
51   quetes      Quete[]
52 }
53
54 model Quete {
55   id          Int          @id @default(autoincrement())
56   titre       String
57   description  String?
58   difficulte   Difficulte  @default(NORMALE)
59   statut       StatutQuete @default(DISPONIBLE)
60   recompenseOr Int         @default(50)
61   createdAt    DateTime    @default(now())
62   // Relation 1-N : une quete appartient a un aventurier
63   aventurierId Int
64   aventurier   Aventurier  @relation(fields: [aventurierId], references: [id])
65 }

```

Pourquoi ces choix ? Les `enum` garantissent que `classe`, `rang`, `difficulte` et `statut` ne peuvent prendre que des valeurs prévues (impossible d'avoir un rang « banane »). Les `@default(...)` évitent d'avoir à tout fournir à la création. Le `?` (ex. `description String?`) rend le champ **optionnel**.

4. Quête 4 — Le grimoire de connexion (`utils/prisma.ts`)

[100 XP]

Tout le projet partagera **une seule** instance du client Prisma. Créez le fichier `utils/prisma.ts` avec **exactement** ce contenu (c'est le pattern standard du cours, avec l'adaptateur Neon et les logs activés) :

Listing 5 – `utils/prisma.ts`

```
1 import { PrismaNeon } from "@prisma/adapter-neon"
2 import { PrismaClient } from "../generated/prisma/client"
3 import dotenv from "dotenv"
4 dotenv.config();
5
6 const adapter = new PrismaNeon({
7   connectionString : process.env.DATABASE_URL,
8 })
9
10 const prisma = new PrismaClient({
11   adapter,
12   log: ['query', 'info', 'warn', 'error'],
13 })
14
15 export default prisma;
```

Piège classique ! L'import du client se fait depuis `../generated/prisma/client` (le dossier défini par `output` dans le schéma) et **non** depuis `@prisma/client`. Si vous vous trompez, vous aurez une erreur `Cannot find module`.

5. Quête 5 — Invoquer les tables (migration & génération)

[150 XP]

À faire

1. Créez les tables dans la base et générez le client :

Listing 6 – Migration

```
1 # Cree la migration + applique le SQL + genere le client
2 npx prisma migrate dev --name init
3
4 # (a relancer apres CHAQUE modification du schema)
5 npx prisma generate
```

Vérifiez l'état à tout moment avec `npx prisma migrate status`. Si Neon répond « `Can't reach database server` », votre base gratuite s'est probablement **mise en pause** : ouvrez le dashboard Neon pour la réveiller.

6. Quête 6 — Peupler le royaume (données réelles)

[200 XP]

À faire (au choix : A ou B)

Insérez au moins **3** aventuriers et **5** quêtes réparties entre eux.

Méthode A — Prisma Studio (interface graphique)

```
1 npx prisma studio
```

Une page s'ouvre sur <http://localhost:5555>. Ajoutez des enregistrements dans **Aventurier**, puis dans **Quete** (en renseignant `aventurierId` avec l'id d'un aventurier existant).

Méthode B — Script de seed (reproductible)

Créez `prisma/seed.ts` :

Listing 7 – `prisma/seed.ts`

```
1 import prisma from "../utils/prisma"
2
3 async function main() {
4   const aragorn = await prisma.aventurier.create({
5     data: {
6       pseudo: "Aragorn",
7       email: "aragorn@guilde.qc.ca",
8       classe: "RODEUR",
9       rang: "OR",
10      niveau: 42,
11      piecesOr: 1200,
12      quetes: {
13        create: [
14          { titre: "Escorter la caravane", difficulte: "NORMALE", recompenseOr: 80
15            },
16          { titre: "Traquer le dragon noir", difficulte: "CAUCHEMAR",
17            statut: "EN_COURS", recompenseOr: 999 },
18        ],
19      },
20    })
21
22   const gandalf = await prisma.aventurier.create({
23     data: {
24       pseudo: "Gandalf",
25       email: "gandalf@guilde.qc.ca",
26       classe: "MAGE",
27       rang: "LEGENDAIRE",
28       niveau: 99,
29       quetes: { create: [{ titre: "Briser le sceau ancien", difficulte: "DIFFICILE
30         " }] },
31     },
32   })
33
34   console.log("Aventuriers crees :", { aragorn: aragorn.id, gandalf: gandalf.id })
35 }
36
37 main()
38   .catch(console.error)
39   .finally(() => prisma.$disconnect())
```

```
1 npm run seed
```

Relancer le seed une 2^e fois provoquera une erreur `Unique constraint failed` sur `email/pseudo` (ils existent déjà). C'est **normal** : changez les valeurs ou videz les tables dans Prisma Studio.

7. Quête 7 — Bâtir la forteresse (le serveur Express)

[300 XP]

On structure le backend proprement : un `server.ts` qui monte un **router** dédié aux aventuriers.

À faire

1. Créez `src/routes/aventuriers.routes.ts` avec les routes GET `/`, GET `/:id`, POST `/` et DELETE `/:id`.
2. Créez `src/server.ts` qui monte ce router sur `/aventuriers`.

Listing 8 – `src/routes/aventuriers.routes.ts`

```

1 import { Router, Request, Response } from "express"
2 import prisma from "../../utils/prisma"
3
4 const routerAventurier = Router()
5
6 // GET tous les aventuriers (avec leurs quetes)
7 routerAventurier.get("/", async (req: Request, res: Response) => {
8   const aventuriers = await prisma.aventurier.findMany({
9     include: { quetes: true },
10   })
11   res.json(aventuriers)
12 })
13
14 // GET un aventurier par id -> http://localhost:3000/aventuriers/1
15 routerAventurier.get("/:id", async (req: Request, res: Response) => {
16   const id = Number(req.params.id)
17   const aventurier = await prisma.aventurier.findUnique({
18     where: { id },
19     include: { quetes: true },
20   })
21   if (!aventurier) {
22     return res.status(404).json({ erreur: 'Aventurier ${id} introuvable' })
23   }
24   res.json(aventurier)
25 })
26
27 // POST creer un aventurier -> body JSON : { "pseudo": "...", "email": "..." }
28 routerAventurier.post("/", async (req: Request, res: Response) => {
29   try {
30     const { pseudo, email, classe, rang } = req.body
31     const aventurier = await prisma.aventurier.create({
32       data: { pseudo, email, classe, rang },
33     })
34     res.status(201).json(aventurier)
35   } catch (e) {
36     res.status(400).json({ erreur: "Creation impossible (pseudo/email manquant ou
37       deja pris ?)" })
38   }
39 })
40
41 // DELETE supprimer un aventurier -> http://localhost:3000/aventuriers/1
42 routerAventurier.delete("/:id", async (req: Request, res: Response) => {
43   const id = Number(req.params.id)
44   try {
45     const aventurier = await prisma.aventurier.delete({ where: { id } })
46     res.json(aventurier)
47   } catch (e) {
48     res.status(404).json({ erreur: 'Aventurier ${id} introuvable' })
49   }
50 })

```

```

50
51 export default routerAventurier

```

Listing 9 – src/server.ts

```

1 import express, { Request, Response } from "express"
2 import dotenv from "dotenv"
3 import routerAventurier from "../routes/aventuriers.routes"
4
5 dotenv.config()
6
7 const app = express()
8 app.use(express.json())           // OBLIGATOIRE avant les routes (parse le body
   JSON)
9
10 app.use("/aventuriers", routerAventurier)
11
12 const PORT = process.env.PORT || 3000
13 app.listen(PORT, () => {
14   console.log(`Serveur sur http://localhost:${PORT}`)
15 })

```

Deux pièges à connaître :

- `req.params.id` est une **string** : convertissez-la avec `Number()` avant de la donner à Prisma (le champ `id` est un `Int`).
- Supprimer un aventurier qui a encore des **quêtes** échouera (contrainte de clé étrangère). Pour permettre la suppression en cascade, ajoutez `onDelete: Cascade` sur la relation côté `Quete` puis refaites `npx prisma migrate dev`.

Lancez le serveur :

```

1 npm run dev

```

Vous devez voir : Serveur sur `http://localhost:3000`.

8. Quête 8 — Mettre l'API à l'épreuve (tests)

[200 XP]

Laissez le serveur tourner, puis testez les routes.

Dans le navigateur (GET uniquement)

URL	Résultat attendu
<code>localhost:3000/aventuriers</code>	La liste JSON des aventuriers + leurs quêtes
<code>localhost:3000/aventuriers/1</code>	L'aventurier d'id 1
<code>localhost:3000/aventuriers/9999</code>	404 + message d'erreur

Avec un fichier `test.rest` (extension REST Client de VS Code)

Listing 10 – test.rest

```

1 GET http://localhost:3000/aventuriers
2
3 ###
4
5 GET http://localhost:3000/aventuriers/1

```

```

6
7  ###
8
9  POST http://localhost:3000/aventuriers
10 Content-Type: application/json
11
12 {
13     "pseudo": "Legolas",
14     "email": "legolas@guilde.qc.ca",
15     "classe": "RODEUR",
16     "rang": "ARGENT"
17 }
18
19  ###
20
21 DELETE http://localhost:3000/aventuriers/1

```

Avec PowerShell

Listing 11 – PowerShell

```

1  # GET tous
2  Invoke-RestMethod http://localhost:3000/aventuriers | ConvertTo-Json -Depth 5
3
4  # POST : creer un aventurier
5  Invoke-RestMethod -Method Post -Uri http://localhost:3000/aventuriers '
6      -ContentType "application/json" '
7      -Body '{ "pseudo": "Gimli", "email": "gimli@guilde.qc.ca", "classe": "GUERRIER"
8              }'
9
10 # DELETE : supprimer l'aventurier d'id 2
11 Invoke-RestMethod -Method Delete -Uri http://localhost:3000/aventuriers/2

```

9. Quête 9 (Boss final) — Prouver la persistance

[200 XP]

Défi

Démontrez que vos données sont **réelles** (en base) et pas en mémoire :

1. Créez un aventurier via POST.
2. **Arrêtez** le serveur (Ctrl+C) puis relancez `npm run dev`.
3. Refaites GET `/aventuriers` : l'aventurier est toujours là → il vient bien de PostgreSQL.
4. Contre-vérifiez dans Prisma Studio que la ligne existe.

Quêtes bonus (XP supplémentaire)

- [+150 XP] Ajoutez un router `quetes.routes.ts` monté sur `/quetes`, avec une route GET `/quetes?statut=DISPONIBLE` qui filtre par statut.
- [+150 XP] Ajoutez une route PUT `/aventuriers/:id` pour faire *monter de niveau* un aventurier (incrémenter `niveau` et `piecesOr`).
- [+100 XP] Activez `onDelete: Cascade` pour que supprimer un aventurier supprime aussi ses quêtes.

- **[+200 XP]** Ajoutez une route qui renvoie le *classement* des aventuriers triés par `piecesOr` décroissant (`orderBy`).

Barème & rangs

XP total	Rang atteint	
0 – 600	Bronze	Apprenti du Royaume
601 – 1100	Argent	Développeur confirmé
1101 – 1600	Or	Architecte de la Guilde
1601 +	Légendaire	Maître du Web ★

Bonne aventure, et que vos requêtes soient toujours en statut 200 OK !